

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS — 1ère année — Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques spécifiques

- À l'issue de cette séance, l'étudiant est capable de :
- Structurer un projet Python en modules (main.py, fonctions.py, donnees.csv)
 - Gérer les erreurs d'exécution avec try/except
 - Lire et écrire des fichiers CSV avec le module csv
 - Développer un générateur de fiches de révision à partir d'un CSV
 - Développer un calculateur de moyenne avec export CSV
 - Présenter son projet selon la grille d'évaluation de la soutenance

Plan du cours

1. Rappel Python — parallèle pseudo-code ↔ Python — 10 min
2. Structure d'un projet Python — 10 min
3. Gestion d'erreurs (try/except) — 15 min
4. Module csv (lire/écrire) — 10 min
5. Projet : Générateur de fiches de révision — 20 min
6. Projet : Calculateur de moyenne avec export CSV — 15 min
7. Conseils soutenance et grille d'évaluation — 10 min

1. Rappel Python (10 min)

Concept	Pseudo-code	Python
Variable	<code>age ← 20</code>	<code>age =</code>
Condition	<code>SI age >= 18</code>	<code>if age</code>
Boucle pour	<code>POUR i DE 0 À 4</code>	<code>for i in</code>
Boucle tant que	<code>TANT QUE x > 0</code>	<code>while</code>
Fonction	<code>FONCTION f(x):</code>	<code>def f(</code>
Retour	<code>RETOURNER x</code>	<code>return</code>
Tableau	<code>t[1]</code> (indice 1)	<code>t[0]<</code>
Affichage	<code>AFFICHER x</code>	<code>print(</code>
Saisie	<code>LIRE x</code>	<code>x = in</code>

Attention : les listes Python commencent à l'indice 0, contrairement au pseudo-code qui commence souvent à 1.

2. Structure d'un projet Python (10 min)

```
graph TD
    ROOT[projet_pedagogique/] --> MAIN[main.py]
    ROOT --> FONC[fonctions.py]
    ROOT --> CSV[donnees.csv]
    MAIN -->|importe| FONC
    FONC -->|lit/écrit| CSV
    style ROOT fill:#34495e,color:#fff
    style MAIN fill:#e74c3c,color:#fff
    style FONC fill:#3498db,color:#fff
    style CSV fill:#2ecc71,color:#fff

    projet_pedagogique/
    |— main.py # Point d'entrée
    |— fonctions.py # Fonctions métier
    |— donnees.csv # Données
    |— README.txt # Documentation

    # fonctions.py
    def calculer_moyenne(notes): return sum(notes)/len(notes)

    def mention(m): return "TB" if m>=16 else "B" if m>=14 else "AB" if m>=12 else "P" if m>=10
    else "NV"
```

3. Gestion d'erreurs — try/except (15 min)

```
graph TD
    DEBUT([Début]) --> TRY["try: note = float(input())"]
    TRY --> VERIF["0 <= note <= 20 ?"]
    VERIF -->|Oui| OK[Afficher]
    VERIF -->|Non| RV["raise ValueError"]
    RV --> EXCEPT["except ValueError:"]
    TRY -->|erreur| EXCEPT
    EXCEPT --> ERR[Message]
    OK --> FIN; ERR --> FIN
    style TRY fill:#3498db; style EXCEPT fill:#e74c3c

    while True:
    try:
    note = float(input("Note (0-20) : "))
    if 0 <= note <= 20:
    break
    print("La note doit être entre 0 et 20.")
    except ValueError:
    print("Veuillez entrer un nombre.")

    try:
    with open("notes.csv", "r") as f:
    data = f.read()
    except FileNotFoundError:
    print("Fichier introuvable.")
    except PermissionError:
    print("Permission refusée.")
```

Règles : spécifier le type d'exception (ValueError, FileNotFoundError), proposer un message clair, ne jamais utiliser except: nu.

4. Module csv (10 min)

```
import csv

# Lecture (liste de listes)
with open("notes.csv", "r", encoding="utf-8") as f:
    lecteur = csv.reader(f)
    next(lecteur) # Ignorer en-tête
    for ligne in lecteur:
        nom, note1, note2 = ligne
        print(f"{nom}: {note1}, {note2}")

# Lecture (dictionnaires)
with open("notes.csv", "r", encoding="utf-8") as f:
    lecteur = csv.DictReader(f)
    for ligne in lecteur:
        print(ligne["nom"], ligne["note1"])

# Écriture
with open("resultats.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["Nom", "Moyenne", "Mention"])
    writer.writerow(["Ali", 15.5, "Bien"])
    writer.writerow(["Sara", 18.0, "Très bien"])

# Écriture avec DictWriter
with open("resultats.csv", "w", newline="", encoding="utf-8") as f:
    colonnes = ["Nom", "Moyenne", "Mention"]
    writer = csv.DictWriter(f, fieldnames=colonnes)
    writer.writeheader()
    writer.writerow({"Nom": "Ali", "Moyenne": 15.5, "Mention": "Bien"})
```

Important : toujours `newline=""` en écriture (évite les doubles interlignes sous Windows).

5. Projet — Générateur de fiches de révision (20 min)

Étape 1 : revision.csv

```
Concept,Définition
Variable,Boîte qui stocke une valeur en mémoire
Fonction,Bloc de code réutilisable
Boucle,Répète un bloc d'instructions
Condition,Exécute selon une expression booléenne
Pile,Structure LIFO (dernier ajouté = premier retiré)
File,Structure FIFO (premier ajouté = premier retiré)
DNS,Traduit les noms de domaine en adresses IP
```

Étape 2-3 : fonctions.py + main.py

```
import csv, random

def charger_fiches(f):
    try:
        with open(f, encoding="utf-8") as fh:
            return list(csv.DictReader(fh))
    except FileNotFoundError:
        print(f"Fichier introuvable.")
    return []

def lancer_quiz(fs):
    if not fs: return
    random.shuffle(fs); score = 0
    for f in fs:
        print(f"\nConcept : {f['Concept']}")
```

```

input("Entrée..."); print(f"Définition : {f['Définition']}")
if input("Correct ? (o/n): ")=="o": score+=1
print(f"Score: {score}/{len(fs)}")

if __name__ == "__main__":
    lancer_quiz(charger_fiches("revision.csv"))

```

6. Projet — Calculateur de moyenne (15 min)

```

import csv

def saisir_notes():
    eleves = []
    print("\n--- SAISIE DES NOTES ---")
    while True:
        nom = input("\nNom (ou 'fin') : ").strip()
        if nom.lower() == "fin": break
        notes = []
        for i in range(1, 4):
            while True:
                try:
                    n = float(input(f" Note {i} (0-20) : "))
                    if 0 <= n <= 20:
                        notes.append(n); break
                    print("Note invalide.")
                except ValueError:
                    print("Entrez un nombre.")
            moy = round(sum(notes)/len(notes), 2)
            eleves.append({"nom": nom, "notes": notes, "moyenne": moy})
            print(f"→ {nom}: {moy:.2f}")
        return eleves

def stats(eleves):
    if not eleves: return
    moyennes = [e["moyenne"] for e in eleves]
    print(f"\nEffectif: {len(eleves)} | Classe: {sum(moyennes)/len(moyennes):.2f}")
    print(f"Max: {max(moyennes):.2f} | Min: {min(moyennes):.2f}")

def exporter(eleves, fichier="resultats.csv"):
    with open(fichier, "w", newline="", encoding="utf-8") as f:
        cols = ["Nom", "Note1", "Note2", "Note3", "Moyenne", "Mention"]
        w = csv.DictWriter(f, fieldnames=cols)
        w.writeheader()
        for e in eleves:
            m = e["moyenne"]
            mention = "TB" if m >= 16 else "B" if m >= 14 else "AB" if m >= 12 else "P" if m >= 10 else "NV"
            w.writerow({"Nom": e["nom"], "Note1": e["notes"][0],
                        "Note2": e["notes"][1], "Note3": e["notes"][2],
                        "Moyenne": f"{m:.2f}", "Mention": mention})
        print(f"Exporté dans {fichier}")

if __name__ == "__main__":
    e = saisir_notes()
    if e: stats(e); exporter(e)

```

7. Conseils soutenance (10 min)

Critère	Poids	Attentes
---------	-------	----------

Fonctionnalité	30 %	Exécution sans erreur
Structure du code	20 %	Fichiers séparés, docstrings
Gestion d'erreurs	15 %	try/except
Données CSV	10 %	Import/export correct
README.txt	10 %	Objectif, utilisation
Présentation orale	15 %	5 min de démo + questions

Soutenance : présentation (30s) → démo (2min30) → code (1min) → améliorations (30s) → questions (30s).

Projet	CSV	Fonctionnalités
Quiz vocabulaire	mots.csv	Tirage aléatoire, score
Tables multiplication	difficultes.csv	Exercices aléatoires
Gestionnaire notes	elevs.csv	Moyennes, export bulletin
Flashcards	concepts.csv	Mode apprentissage

QCM d'auto-évaluation

- Comment importer les fonctions d'un fichier `fonctions.py` ?
a) `include fonctions` b) `from fonctions import *` ✓ c) `using fonctions`
- Quelle exception pour un fichier introuvable ?
a) `ValueError` b) `FileNotFoundError` ✓ c) `TypeError`
- Quel paramètre éviter les doubles interlignes CSV sous Windows ?
a) `encoding="utf-8"` b) `newline=""` ✓ c) `mode="wb"`
- Quelle méthode de `csv` retourne les lignes en dictionnaires ?
a) `csv.reader()` b) `csv.DictReader()` ✓ c) `csv.reader_dict()`
- Pondération de la Fonctionnalité dans la grille ?
a) 20 % b) 30 % ✓ c) 15 %

Conclusion (5 min)

Ce dernier cours vous a donné les outils pour un projet Python complet : modules, gestion d'erreurs, CSV, et soutenance. Vous êtes capables de créer des outils pédagogiques concrets (quiz, générateurs d'exercices, calculateurs de notes) pour vos futures classes.

Félicitations !

Note pédagogique ENS

Bilan : le projet valide l'ensemble des compétences du module. La démarche (spécifications → conception → implémentation → test → soutenance) est celle du Bac NSI.

Prolongements (ENS 2ème année) : POO, Web (Flask), bases de données (SQLite), projet interdisciplinaire.

Bibliographie

- Documentation Python 3 : docs.python.org/fr/3/tutorial/
- Module csv : docs.python.org/fr/3/library/csv.html
- Downey — Think Python (O'Reilly)
- Programme NSI — eduscol.education.fr